META GLOBAL AI DEVELOPER HACKATHON 2026 · INDIE TRACK

ONLY YOU

The handbook: what it is, what it proves, and how to change every part of it.

Muse Spark | Muse Wonder | Muse Voice Transcribe | SAM 3.1 | Muse Image

Capability is solved. Authority isn’t.

Repo · Playground · Verifier

v2026-10-28 · 2d5aa04 · 172 tests · audit 67/67

Contents

Read pages 3–7 for the pitch, architecture and evidence; 8–13 for the demo, run/verify and change maps; 14–18 for the build, submission, limits and appendix.

Why this project exists	3
The authority model	4
The Meta stack	5
What the evidence proves (1)	6
What the evidence proves (2)	7
The demo, beat by beat	8
The presentation kit	9
Run it	10
Verify it	11
How to change the numbers	12
How to change the documents and media	13
How this handbook is built	14
Submission runbook	15
Limitations and roadmap	16
Judge Q&A	17
Appendix	18

Source of this document: `docs/handbook.md`. Layout and style: `tools/handbook.py`. Rebuild: `make handbook`. Every number here has a canonical artifact listed on page 12, so changing one starts there.

STATUS AT BUILD

- **172 tests** · audit **67/67** · CI green (tests + Docker)
- **179 s** film · 1-page brief · 2-page report · 10-slide deck
- **Four Meta models**, each with a committed artifact
- **Frontier DeepSeek:** 50% undefended → **0%** gated, 18/18 receipts
- Frontier DeepSeek: **50%** → **0%**, 18/18 receipts
- Human voice study: owner 3/3 · **0/12 FAR** at capped 0.10 (pilot + held-out)

HOW TO USE THIS BOOK

- Every page ends with **EDIT · BUILD · PROOF**: the file, the command, the artifact.
- Tables are the checkable evidence; prose is the why.
- Change a number by changing its canonical artifact (page 12), never prose first.
- This PDF refuses to build if any fixed page overflows.

Why this project exists

> **Capability is solved. Authority isn't.**
> Everyone is building agents that can do things. This is the one you can hand your wallet.

Your glasses can hear the world. The question is **who they obey**. A wearable agent has a camera, a microphone and (soon) a wallet, so the world can reach its context: a TV ad, a poster, a webpage, a look-alike invoice. Meta's own WebMCP developer documentation says the untrusted-content hint is *"not an enforced security boundary."* The model cannot be the boundary, because the model is what is being confused.

ONLY YOU is a provenance-gated authority layer for wearable agents. Only the enrolled owner's voice is a command. Scene text, page text and documents are **data**: read, summarised, never obeyed. Irreversible actions need a signed, scoped, expiring **mandate**. Every action *and every refusal* leaves a verifiable receipt.

It is built on **four Meta models** through the Meta Model API — Muse Spark (reasoning), Muse Wonder (scene reading), Muse Voice Transcribe (spoken commands) and SAM 3.1 (object grounding) — plus Muse Image for demo assets.

What it is not

- Not a better prompt, and not an alignment claim. It is a deterministic gate outside the model.
- Not a biometric product: the voice check is a channel demonstrator, not anti-spoofing.
- Not a real payment system: the merchant, invoices and ledger are a mock world.
- Not a claim that prompt injection is solved. It is a claim that authority can be enforced even when the model is helpful to the attacker.

The one-sentence architecture

Sensors → **label layer** (who produced this?) → **model proposal** → **constitution + mandates** → execute or refuse → **signed receipt**. The model proposes; the gate decides; the ledger proves.

Why Meta

Meta's two big bets are **glasses** and **agentic commerce**. Both need the same missing layer: a merchant cannot verify an agent's authority, and a wearable cannot tell its owner from the world. A signed mandate turns the commerce problem from *"which agent is this?"* into *"verify this signature, payee, cap and expiry offline"*. That is a trust layer Meta's products need, and it is the argument for building it inside Meta rather than beside it.

> **25% → 0%** the confused-deputy corpus with the full gate

EDIT	BUILD	PROOF
README.md · docs/brief.md	make brief	docs/brief.pdf

The authority model

The gate is deliberately boring: a fixed, auditable constitution that runs before any tool is touched. It does not decide whether a document is trustworthy; it decides based on **provenance**, **policy** and **delegation**.

Channels and what they can do

Channel	Example	Can it command?
`voice` (enrolled owner)	“pay the electrician 4,500”	**Yes** — after speaker check
`scene`	poster, TV ad, camera text	No — data only
`document`	invoice text, PDF, email	No — data only (can supply payee/amount)
`web`	page text, DOM, hidden text	No — data only
`model`	proposal / tool call	No — it is a proposal, not authority

The constitution (defaults)

Rule	Decision
`R0-source-not-authoritative`	Deny an action whose cause is not an authoritative channel
`R1-payee-not-contact`	Deny paying an unknown payee
`R2-payment-over-limit`	Ask above the per-action limit; also caps mandate amount
`R6-mandate`	Allow only inside a valid signed mandate (payee, cap, expiry, remaining)
`R7-policy-change`	Every policy edit is itself receipted and signed

Mandates

A mandate is an Ed25519-signed statement: *owner key → agent key, payees, max amount, expiry, optional period/sub-limit/parent*. A merchant verifies it **offline** with only the public key — no platform call, no account, no API. Delegation is bounded: sub-mandates cannot widen the parent, and scheduled instructions still pass the same gate.

Receipts

Every action and every refusal appends a receipt: rule, reason, evidence, action payload, source labels, mandate, timestamp and previous hash. Receipts form an HMAC chain; the audit export includes the chain head, spend totals, mandates and public key. `tools/verify_audit.py` and the browser verifier recompute the chain and check mandate signatures offline; one changed field breaks verification.

The important sentence: the model never sees raw unlabelled content, and never holds authority. It receives labelled context, proposes an action, and the gate re-derives whether that action is allowed.

0% / 0% attacks executed / false refusals at the full gate

EDIT
src/onlyyou/provenance.py · policy.py · mandates.py · receipts.py

BUILD
make test

PROOF
results-real-doc/

The Meta stack

Four models do the perception and reasoning; none of them is trusted with authority.

Model	Role in the demo	Endpoint / call	Committed artifact
Muse Spark 1.3	proposes the action; gate re-validates	`/v1/chat/completions`	`results-real*/` (four real runs)
Muse Wonder 1.0	camera frame → text + approximate boxes	`/v1/chat/completions` with image	`results/scene-muse.json` (15/15, 0.0% CER)
Muse Voice Transcribe 1.0	spoken command → transcript	`POST /v1/asr/transcribe`	`results/voice-asr.json`, `results/voice-e2e.json`
SAM 3.1	object grounding on the scene frame	`POST /v1/responses`	`results-sam/sam.json` (paper → 1 box)
Muse Image 1.0	demo posters, TV frame, invoice, article, hero	image generation	`src/onlyyou/web/fakecam-*.jpg`

What is measured, and what is not

- **SAM 3.1:** a short noun phrase (`paper` , `document`) returns a box in source pixels; the live overlay draws it. Prompts like `text` / `invoice` return **zero regions** — SAM segments objects, not text. Mask payloads are returned but **not decoded**; text-region crops are roadmap. No fake text-segmentation claim is made.
- **Muse Voice Transcribe:** real ASR, imperfect. The committed sample transcribes to *“Pay the electrician electrician 4,500 rupees.”* — one duplicated word. That imperfect transcript still has to pass the voice check and the gate; `make voice-e2e` proves it executes under **R6** with a receipt.
- **Muse Wonder:** 15 synthetic signs, 100% detection, mean CER 0.0%, approximate boxes; tesseract is the offline fallback.
- **Muse Spark:** the real-model runs are the evidence for the gate; see pages 6–7.

Why this matters for the pitch

The project is not “a wrapper around one model”. It is **the authority layer the wearable stack needs**, demonstrated across four Meta models with different failure modes. The gate and the receipts are the same for all of them; the model is replaceable.

4 models one authority layer; each model has a committed artifact

EDIT	BUILD	PROOF
src/onlyyou/model.py · scene_muse.py · sam.py · voice_muse.py	make sam-check · asr-check	results-sam/ · results/voice-asr.json

What the evidence proves (1)

The confused-deputy corpus (the hard case)

The owner asks to pay a real invoice; a document supplies the payee. The model is **helpful and wrong**. Muse Spark 1.3, 8 attacks / 12 benign, document channel:

Arm	Architecture	Attack success	False refusals	Receipts
A	undefended	**25.0%**	0%	—
B	prompt-only defences	**37.5%**	0%	—
C	provenance labels, no gate	**50.0%**	0%	—
D	**full ONLY YOU**	**0.0%**	**0%**	**18/18 verified**

The full gate is the only arm with zero executed attacks, and it does not buy that with false refusals. Contributor build on the same corpus: **50%** → **37.5%** → **50%** → **0%**. Final-code reproduction: every arm **0%**, **16/16 receipts**, chain intact, 99 live calls, p50 3.1 s per gated run.

The general corpus (alignment is the first line)

20 attacks / 20 benign across ambient audio, scene, voice and web: **every arm executed 0%**. Muse Spark refused even undefended. That is the honest picture and the reframe: **alignment is the first line; the confused deputy is the gap.** The gate earns its place where the model is trying to be helpful, not where a prompt is obviously malicious.

What a judge should take from this

- The result is conditional on **correct labels**. The label layer is measured and its failures are published (81% DOM recall, 14.3% false positives; adaptive corpus 20% executed).
- The sample is small and synthetic; the numbers are reported with that caveat, not inflated.
- The same result is reproduced on final code and on a second model build, with receipts.

18/18 receipts verified on the hard corpus; 99 live calls

EDIT	BUILD	PROOF
results-real-doc/benchmark.json	make bench-meta	results-real-verify/benchmark.json

What the evidence proves (2)

Cross-model audit

`make cross-model` runs the same two scenarios through every configured model and exports one artifact per model sharing a device key. The browser verifier checks both chains and both mandates in one pass and reports **VERIFIED · CROSS-MODEL**.

Model	Benign invoice	Look-alike invoice	Receipts
Muse Spark 1.3	executed · `R6`	blocked · `R0-source-not-authoritative`	2/2 verified
DeepSeek-chat (frontier)	executed · `R6`	blocked · `R0-source-not-authoritative`	2/2 verified

Frontier experiment — the headline

`make frontier-eval` ran the full **8 attacks / 12 benign** confused-deputy corpus through the unchanged gate with **DeepSeek-chat** (frontier-class, temperature 0; `results-frontier/`):

Arm	Attack success	False refusals	Receipts
A — undefended	**50.0%** (4/8 attacks; 10/12 benign paid)	0%	—
D — full gate	**0.0%**	**0.0%**	**18/18 verified**

40 calls, 54 s. The more capable the model, the more it needs an authority layer. A local 3B plumbing run is kept in `results-local-llama/` as a secondary check, not the frontier claim.

Other measurements

Test	Result
Forged sensor envelopes	**6/6 executed without → 0/6 with** a signed bus
Adaptive corpus (attacker controls signals)	label error 80%, **executed 20%** (3/15; owner-confirmed cases)
DOM hidden-text detector	recall **81.0%**, FPR **14.3%**, 8 hiding techniques missed
Browser computed-style labeler	42/42 hidden, 0/35 false positives (upper bound, written after cases)
Human voice study (LibriSpeech)	owner 3/3; **0/12 impostors** at capped 0.10 on pilot and 12 unseen speakers; uncapped pilot 2/12
Synthetic TTS study	**6/6 impostors rejected**, owner 3/3
Frontier (DeepSeek-chat)	undefended **50%** → gate **0%**, 0% false refusals, **18/18 receipts**
Latency	constitution **0.001 ms**, dry-run **4.3 ms**, HTTP command **12.8 ms**, OCR **156 ms**
Latency end-to-end	6.8–11.9 s p50, dominated by model inference

50% → 0% frontier DeepSeek: 50% → 0%; human FAR 0/12 capped

EDIT
results-cross-model/ · results/voice-study-human.json

BUILD
make cross-model · voice-study

PROOF
results/latency.json

The demo, beat by beat

The 3-minute film is **179 s** (2:59), narrated and captioned. It is built around contrast, not features.

Opening (0:00–0:15): the contrast

Four seconds of cards before the film proper:

1. **Undefended · DeepSeek-chat (frontier-class) — 50% injected invoices PAID** (4/8 attacks).
2. **Same model · full ONLY YOU gate — 0% paid · 18/18 receipts verified**.

This is the feedback's hook executed with a frontier-class model: the model obeys the document, the gate obeys the owner. The label and the artifact (`results-frontier/`) are explicit.

Chapters

Time	Beat	What it proves
0:04	Who does your agent answer to	The premise
0:16	The gap	Meta's own "not an enforced boundary"
0:32	Meta's own warning	The world reaches the context
0:48	A mandate, then paid without a prompt	Delegation with a receipt
1:06	The look-alike invoice is refused	Confused deputy blocked
1:22	A poster is scene data	Provenance routing
1:36	A TV ad is not your voice	Channel check
1:50	Hidden page text	DOM data, never command
2:04	The record	Receipts + ledger context
2:20	Real-model numbers	Evidence
2:40	Close	The thesis

Supporting assets

Asset	What it is
`docs/demo-contrast.mp4`	20.5 s standalone contrast clip
`docs/demo-3min.srt` / `.vtt`	Shifted +4 s for the new opening; captions match
`docs/img/attack-blocked.gif`	26 s README GIF cut from the refusal beat
`docs/demo-fallback.mp4`	82 s silent walkthrough
`docs/img/youtube-thumb.jpg`	1280×720 thumbnail; source `docs/assets/youtube-thumb.html`

179 s a four-second contrast opens the film

EDIT	BUILD	PROOF
docs/narration-script.md · demo-3min.mp4	make voice · audit	docs/demo-3min.mp4 (179 s)

The presentation kit

Every judge-facing asset follows one arc: **hook** → **problem** → **demo** → **evidence** → **why Meta** → **honesty**.

README (the first two minutes)

The order is deliberate: one-liner and thesis → 30-second paragraph → GIF → **evidence table** → one-command `make judge` → assets → architecture and details. The evidence table includes the human voice result and the latency numbers, so the honest numbers are above the fold, not buried.

Documents

Asset	Size	Source	Rebuild
`docs/brief.pdf`	**1 page**	`docs/brief.md`	`make brief`
`docs/report.pdf`	**2 pages**	`docs/report.md`	`make report`
`docs/pitch.pdf`	**10 slides**	`docs/pitch-event.md`	`make pitch`
`docs/handbook.pdf`	this document	`docs/handbook.md`	`make handbook`

Video and social

- `docs/demo-3min.mp4` — 179 s, narrated + captioned, contrast opening.
- `docs/demo-contrast.mp4` — 20.5 s standalone contrast.
- `docs/demo-task-clip.mp4` — 20 s agentic task trace.
- 45 s vertical cut, X posts and share cards in `docs/x-post.md` and `docs/img/x-*.png`.
- Upload kit with title/description/chapters/captions: `docs/youtube-upload.md`.

The pitch in one slide

> The more capable the model, the more it needs an authority layer. Frontier models obey the world; ONLY YOU makes them obey the owner.

1 · 2 · 10 brief pages · report pages · deck slides

EDIT	BUILD	PROOF
docs/brief.md · report.md · pitch-event.md	make brief report pitch thumb	docs/*.pdf

Run it

Everything is stdlib Python plus image/audio command-line tools; the Docker image is the clean path.

```
make judge        # 172 tests + 67-check audit + live server smoke (about 25 s)
make preflight    # live links, repo visibility, submission files
make web          # http://127.0.0.1:8000/?script=full&fakecam=1
```

No Python at all:

```
docker build -t only-you-demo .
docker run --rm -p 8000:8000 only-you-demo
# or: docker compose up --build
```

Environment

Variable	Purpose
`META_API_BASE` / `META_API_KEY`	real Muse models; without them a simulated model runs and the report says so
`MUSE_MODEL`	default `muse-spark-1.3`
`SAM_MODEL`	default `sam-3.1` ; `SAM_PROMPT` default `paper`
`FRONTIER_` / *`OPENAI_`* / `GEMINI_` / *`ANTHROPIC_`* / `DEEPSEEK_` / *`OPENROUTER_`*	the frontier adapter
`ONLYYOU_MODEL=simulated`	force offline mode

Experiments (each writes a committed artifact)

```
make bench            # full four-arm benchmark (simulated or real)
make frontier-eval    # A vs D on the confused-deputy corpus (API key or local Ollama)
make cross-model      # Muse + DeepSeek, one audit bundle
make sam-check        # SAM 3.1 object boxes on the demo frame
make asr-check        # Muse Voice Transcribe on the committed sample
make voice-e2e        # ASR → voice check → gate → receipt
make voice-study      # any owner/impostor folders
make latency          # routing/gate/OCR/HTTP latency
make links            # local link guard
```

The playground

`docs/playground.html` is static: it works on GitHub Pages with no server and no install. The live demo (`make web`) adds real model calls, the camera feed, the merchant terminal and the verifier.

make judge 172 tests, 67-check audit and a live smoke in ~25 s

EDIT	BUILD	PROOF
Makefile · tools/	make judge · preflight	AUDIT.md · results/

Verify it

Do not trust the UI; verify the evidence. This is the shortest path a judge can take.

Browser, no install, no Python

1. Open <https://sushant-me.github.io/only-you/verify.html>.
2. Click **load cross-model bundle**.
3. The page recomputes every receipt HMAC and every Ed25519 mandate signature in every model bundle and reports **VERIFIED · CROSS-MODEL** with per-model rows.

The same page can load `docs/audit-sample.json` (single bundle) and shows what fails if one byte changes. The HMAC secret is public **by design** in the demo; the mandate signatures are real Ed25519.

Command line

```
python3 tools/verify_audit.py results-cross-model/cross-model-audit.json
python3 tools/verify_audit.py results-real-doc/audit.json
```

What to tamper with (proof the chain is real)

- Change one amount in `docs/audit-sample.json` → receipt signature mismatch.
- Change one mandate cap → Ed25519 verification fails.
- Reorder two receipts → chain link mismatch.

Reproducibility

```
git clone https://github.com/sushant-me/only-you
cd only-you && make judge && make preflight
```

`make judge` runs the tests, the 67-check self-audit (including page counts, video durations, committed numbers) and a live-server smoke test. It is the one command to put in front of a sceptical judge.

VERIFIED CROSS-MODEL: both chains and both mandates, in the browser

EDIT	BUILD	PROOF
tools/verify_audit.py · docs/verify.html	make cross-model	results-cross-model/cross-model-audit.json

How to change the numbers

Never edit a number in prose first. Every claim has a canonical artifact; change the artifact or the run, then propagate. The self-audit checks several of these and will fail if the docs drift.

Claim	Canonical source	Also quoted in	Regenerate with
172 tests	`make test` output / CI	README, brief, report, deck, handbook	`make test`
67/67 audit	`AUDIT.md`	README, brief, deck, handbook	`python3 tools/audit.py`
179 s video	`docs/demo-3min.mp4`	README, YouTube kit, submission	re-cut + `tools/audit.py`
25 / 37.5 / 50 / 0%	`results-real-doc/benchmark.json`	README, brief, report, deck	`make bench-meta` (needs key)
6/6 → 0/6 forged	`results/forged-sensors.json`	README, report, deck	`make scenes` / test suite
81.0% / 14.3%	`results/dom-labeler.json`	report, limitations	`make scenes`
Human FAR 0/12	`results/voice-study-human-heldout.json`	README, brief, report, deck	study protocol in `docs/voice-study.md`
Frontier A vs D 50% → 0%	`results-frontier/benchmark.json`	README, report, brief, deck	`make frontier-eval`
Latency table	`results/latency.json`	README, report, handbook	`make latency`
SAM paper box	`results-sam/sam.json`	docs/sam.md, handbook	`make sam-check`
ASR transcript	`results/voice-asr.json`	docs/voice-study.md	`make asr-check`
Cross-model audit	`results-cross-model/cross-model-audit.json`	README, verify page sample	`make cross-model`
Brief/report/deck pages	`docs/*.pdf`	README, submission, audit	`make brief report pitch`

Consistency checklist after changing a number

```
grep -rn "172 tests\|67/67\|50%\|0/12\|179" README.md docs/*.md \
  docs/pitch-event.md SUBMIT-NOW.md ../FORM-ANSWERS.md
make brief report pitch handbook
python3 tools/audit.py          # must be 67/67
make preflight                  # live links and pages
```

If the change is in a benchmark, commit the new `results*/` directory **and** regenerate `AUDIT.md`; if it is in the video, update SRT/VTT, chapters and the audit duration check.

67/67 audit checks that catch documentation drift

EDIT
this table → canonical artifacts

BUILD
python3 tools/audit.py

PROOF
AUDIT.md (67/67)

How to change the documents and media

Asset	Source you edit	Build command
README	`README.md`	— (GitHub renders)
1-page brief	`docs/brief.md`	`make brief`
2-page report	`docs/report.md`	`make report`
10-slide deck	`docs/pitch-event.md`	`make pitch`
This handbook	`docs/handbook.md`	`make handbook`
YouTube thumbnail	`docs/assets/youtube-thumb.html`	`make thumb`
README GIF	`docs/demo-3min.mp4` (refusal beat)	`ffmpeg -ss 79 -t 26 …`
Video	`docs/narration-script.md` + captions	`make voice VOICE=…` / re-cut
Captions	`docs/demo-3min.srt` / `.vtt`	shift by the same offset as the cut
X posts	`docs/x-post.md`	—
Judge Q&A	`docs/judge-qa.md`	—
Submission sheet	`docs/submission-final.md`	—

Style and layout

- PDF style tokens live in `tools/md_to_pdf.py` (report/brief/deck) and `tools/handbook.py` (this document).
- The handbook uses fixed A4 pages, so it refuses to print if a page overflows; run `python3 tools/handbook.py --check-only` and trim the page.
- To add a handbook page, insert a page-break comment and a `# Title`; the contents and page numbers update automatically.
- To change a video beat: edit `docs/narration-script.md`, regenerate narration, re-mux, then shift the SRT/VTT and chapter list by the same amount.

The visual language

Dark cover, white interiors, one red accent (`#ff2d55`) for risk/refusal, one green (`#2fd18c`) for verified/allowed, monospace for commands and evidence paths. Keep tables to the numbers that a judge can check; keep the prose about why.

regenerates every asset here has a source and a build command

EDIT	BUILD	PROOF
docs/*.md · docs/assets/	make brief report pitch handbook thumb	docs/handbook.pdf

How this handbook is built

This is a real artifact, not a screenshot of the docs. It is reproducible and editable.

Pipeline

1. `docs/handbook.md` holds the content in Markdown, with a page-break comment between pages.
2. `tools/handbook.py` parses a small Markdown subset (headings, lists, tables, code fences, quotes, images), adds the cover and a contents page with real page numbers, inserts running headers/footers, and renders with headless Chromium.
3. Before printing, the builder opens the HTML with Chromium and measures every page; if any page's content exceeds A4 it prints `OVERFLOW` and refuses to write the PDF.
4. `make handbook` writes `docs/handbook.pdf`.

Why fixed pages

Fixed A4 pages make the handbook look like a designed document rather than a long web page, and they make the table of contents trustworthy. The trade-off is that content must be curated to fit; the overflow checker turns that into a fast failure instead of silent clipping.

Editing workflow

```
# 1. edit docs/handbook.md
python3 tools/handbook.py --check-only   # fast layout check
make handbook                             # writes docs/handbook.pdf
pdfinfo docs/handbook.pdf | head          # page count
```

If a page overflows, move a paragraph to the next page or shorten a table. The footer version is taken from the current git commit and date at build time, so the PDF always states the code it describes.

Where the truth lives

The handbook is a guide, not a second source of truth. When a number changes, change the canonical artifact (page 12), then update this document and rebuild. The audit checks the number-bearing assets; the handbook is checked by `make preflight` for existence.

18 pages fixed A4 pages, overflow-checked before printing

EDIT	BUILD	PROOF
docs/handbook.md · tools/handbook.py	make handbook	docs/handbook.pdf

Submission runbook

Deadline: **4 Nov 2026, 9:00 PM ET / 5 Nov 2026, 7:45 AM NPT**. Live finals: **9 Nov**.

Step 1 — YouTube (10 minutes)

1. Open YouTube Studio → Create → Upload videos.
2. File: `docs/demo-3min.mp4` (179 s, narrated, captioned).
3. Title/description/category: paste from `docs/youtube-upload.md`.
4. Visibility: **Unlisted**.
5. Subtitles: upload `docs/demo-3min.srt`.
6. Thumbnail: `docs/img/youtube-thumb.jpg`.
7. Open the link in a logged-out window; if it plays, copy it.

Step 2 — the form (5 minutes)

1. Open the hackathon application page linked from the event.
2. Paste the long and short answers from `FORM-ANSWERS.md` (personal details in `docs/form-paste.txt`).
3. Put the YouTube URL in the demo field.
4. Project links: repo, live page, playground, verifier, cross-model audit, brief, report, deck, contrast clip.
5. Run `make preflight` before submitting; screenshot the confirmation page.

Final checks

- [ ] Repo public and opens logged out.
- [ ] `make judge` green in a fresh clone.
- [ ] CI green (tests + Docker).
- [ ] Every link opens in an incognito window.
- [ ] Video plays logged out and is ≤ 3:00.
- [ ] Brief = 1 page, report = 2 pages, deck = 10 slides, handbook current.
- [ ] Human FAR, SAM mask limit and ASR errors are stated honestly.

4 Nov 9:00 PM ET / 5 Nov, 7:45 AM NPT

EDIT	BUILD	PROOF
SUBMIT-NOW.md · FORM-ANSWERS.md	make preflight	docs/submission-final.md

Limitations and roadmap

The failure list has **17 rows**; the point is that it exists. The top risks:

Risk	Status	Next step
Cloned voice passes the channel check	**Open**; human FAR **0/12** at capped 0.10 (pilot + held-out); uncapped pilot 2/12	ECAPA/TDNN embedding + liveness/anti-spoofing
Label layer misses hiding techniques	**Open**; 81% recall, 14.3% FPR	browser computed-style labeler + SAM mask decode
Adaptive attacker controls signals	**20% executed** (owner-confirmed cases)	liveness + proximity attestation + confirmations
Evidence coverage on paraphrases	**1/8 receipted**	trigger-verb classifier + always-receipt policy
SAM text masks not decoded	**Open**; object boxes only	decode masks, crop text before OCR
ASR errors	Real; duplicated word in sample	vocabulary biasing + confidence gate
Demo HMAC secret is public	**By design**	production roots in secure element
Frontier result limited to one provider	**DeepSeek-chat: 50%** → **0%** shipped	add GPT/Gemini/Claude key and re-run `make frontier-eval`

Roadmap with Meta

1. WebMCP authority propagation: signed provenance labels on every tool result.
2. Device-rooted mandates (secure element + attestation) so a merchant verifies the device, not just a key.
3. An authority-failure eval across Muse models — the confused-deputy, adaptive and forged-sensor corpora as a reusable benchmark.
4. Muse Wonder in the loop for long-context memory, with the receipt ledger entering as labelled data.

The honest line: the gate is engineered and tested; the perception and evidence layers are measured where they fail. That is what makes it auditable rather than magical.

17 rows of published failure modes, each with a status and a next step

EDIT	BUILD	PROOF
docs/limitations.md · judge-qa.md	make test	docs/limitations.md

Judge Q&A

What is actually new? Provenance is the authority boundary, not the prompt: only an enrolled voice can command, and a signed mandate is what lets an action execute. Receipts cover refusals too.

Does the gate do anything on easy attacks? No — Muse Spark refused even undefended on the general corpus. That is alignment's first line. The gate matters on the confused deputy: 25/37.5/50% → **0%**.

How big is the evidence? Small and synthetic, stated everywhere: 8 attacks/12 benign in the hard corpus, four real-model runs, reproduced on final code (16/16 receipts, 99 calls).

Why four Meta models? Muse Spark proposes, Muse Wonder reads the scene, Muse Voice Transcribe hears the command, SAM 3.1 grounds the object. Each has a committed artifact and a measured limit.

Does SAM segment text? No. It grounds objects (`paper` → 1 box); text prompts return zero and masks are not decoded. Text-region segmentation is roadmap, not claimed.

Is the voice check biometric? No. It is a channel demonstrator: **0/12** human FAR at the capped 0.10 threshold on the pilot and 12 unseen speakers (uncapped pilot 2/12); the cap is a mitigation, not anti-spoofing. Next: ECAPA + liveness.

What about prompt injection? Not solved. The gate enforces authority even when the model is helpful to the attacker; the label layer is the measured weak point.

Can a judge verify without trusting the app? Yes: browser verifier (WebCrypto) or `tools/verify_audit.py`; tamper with one byte and it fails.

What would you build next at Meta? Signed provenance on tool results (WebMCP), device-rooted mandates, and an authority-failure benchmark across Muse models.

Why should this exist inside Meta? Glasses and agentic commerce both need verifiable authority; a signed mandate turns "which agent is this?" into an offline signature check.

Appendix

Repository map

Path	What lives there
`src/onlyyou/`	gate, constitution, mandates, receipts, provenance, models, SAM/ASR adapters, webapp
`tools/`	benchmark, audit, PDF builders, verifiers, sam/asr/voice/latency/handbook tools
`tests/`	172 tests, stdlib `unittest` only
`results*/`	committed benchmark, receipts, cross-model, SAM, ASR, voice studies, latency
`docs/`	brief, report, deck, handbook, playground, verifier, video, images
`.github/workflows/ci.yml`	test job + Docker build/run/probe job

Endpoints a judge can hit

```
GET  /                    demo UI
POST /api/command         the gate
POST /api/frame           scene text + SAM box
POST /api/voice/transcribe Muse Voice Transcribe
POST /api/task            agentic task trace
POST /api/export          signed audit bundle
GET  /verify/<receipt>    per-receipt page
GET  /merchant-page       offline mandate verification
GET  /docs                OpenAPI 3.1 + Swagger
```

Environment and versions

Muse Spark 1.3 · Muse Wonder 1.0 · Muse Voice Transcribe 1.0 · SAM 3.1 · Muse Image 1.0 · Python 3 stdlib · Ed25519 via OpenSSL · tesseract · ImageMagick · ffmpeg · headless Chromium.

Credits and honesty

Synthetic narration is labelled and replaceable; the demo HMAC secret is public by design; SAM masks are not decoded; ASR is imperfect; the human study uses LibriSpeech (CC BY 4.0, no audio committed). Full credits in `NOTICE.md`, failure list in `docs/limitations.md`, security notes in `SECURITY.md`.

Everyone is building agents that can do things. This is the one you can hand your wallet.

1 command make judge reproduces the whole evidence path

EDIT	BUILD	PROOF
repo map · this page	make judge	git log --oneline